

Conteneurisation et déploiement : services, images tierces, histoire MinIO → silo

Le déploiement de DataShare est à **moitié conteneurisé**, et c'est le point à énoncer correctement à l'oral : `deploy/docker-compose.yml` ne lance que les **dépendances** (PostgreSQL, stockage S3, outils). Le backend et le frontend ont chacun leur `Dockerfile`, mais ne sont pas des services du compose. Le second enjeu : les images d'infrastructure ne sont pas surveillées par Dependabot ; elles sont suivies **à la main**, et l'histoire MinIO montre pourquoi ce suivi compte.

Les services

Élément	Où	Rôle
db	compose	postgres:16-alpine, volume db-data, healthcheck avec pg_isready
minio	compose	stockage compatible S3 (API 9000, console 9001), image silo
createbuckets	compose	conteneur jetable qui crée le bucket datashare-files, après minio sain (depends_on avec service_healthy)
adminer	compose	interface web de la base, outil de dev uniquement
backend	backend/Dockerfile	hors compose : build Maven puis JRE 21 Alpine, utilisateur non root
frontend	frontend/Dockerfile	hors compose : build Node 24 puis nginx, qui sert le bundle

Le commentaire du compose le dit : « Le backend et le frontend se lancent hors compose à ce stade. » En dev, le back tourne avec `./mvnw spring-boot:run` ; en prod, `java -jar` avec le profil `prod` (`MAINTENANCE.md`). Le front conteneurisé est lancé par `scripts/start-frontend-docker`.

Builds multi-étapes

```
FROM maven:3.9-eclipse-temurin-21 AS build
...
FROM eclipse-temurin:21-jre-alpine
COPY --from=build /build/target/*.jar app.jar
USER app
```

Une image de **build** (Maven, JDK, sources) sert à compiler ; l'image finale ne contient que le JRE (Java Runtime Environment) et le jar, sous un utilisateur non root (`USER app`) : image plus petite, surface d'attaque réduite. Même schéma côté front : Node compile, nginx sert les fichiers statiques et proxifie `/api/` vers `backend:8080` avec `client_max_body_size 1024m` (limite 1 Go).

Healthcheck et ordre de démarrage

`depends_on` seul attend que le conteneur *démarre*, pas qu'il soit *prêt*. Ici `createbuckets` attend `condition: service_healthy` : le `healthcheck` de `minio` (`mccli ready local`) doit réussir avant de créer le bucket.

Suivi manuel des images tierces

Dependabot (`.github/dependabot.yml`) ne surveille que les `Dockerfile` de `backend/` et `frontend/`. Les images d'infrastructure du compose doivent être suivies **à la main** et rester **alignées à 3 endroits** :

Image	<code>docker-compose.yml</code>	<code>ci.yml</code> (<code>frontend-e2e</code>)	Testcontainers
<code>postgres:16-alpine</code>	oui	oui (service)	via le module PostgreSQL
<code>ghcr.io/denoyela-4913/silo:RELEASE.2026-09-16T00-00-00Z</code>	oui	oui	oui (<code>MinioTestcontainersConfiguration</code>)
<code>adminer:latest</code>	oui	non	non

Pourquoi l'alignement : un tag qui diffère entre dev, CI et tests fait passer les tests contre une version différente de celle utilisée à la main, donc des bugs non reproductibles. Point d'attention relevé dans le compose : `adminer:latest` est **non épinglé**, contrairement aux autres (toléré car outil de dev uniquement, marqué comme tel dans `MAINTENANCE.md`).

L'histoire MinIO → silo

Date (selon <code>MAINTENANCE.md</code>)	Événement	Conséquence
10/2025	<code>minio/minio</code> retiré de Docker Hub (MinIO Community passé <i>source-only</i>)	migration vers <code>quay.io/minio</code>
09/2026	<code>quay.io/minio</code> devenu privé (401)	migration vers <code>pgsty/minio</code>
06/08/2026	<code>pgsty/minio</code> gelé, renommé <code>pgsty/silo</code>	fork <code>denoyela-4913/silo</code>

Historique git correspondant : #52 (Docker Hub → `quay.io`), #57 (`quay.io` → `pgsty/minio`), #86 (fork `silo` sur GHCR, `mccli` remplace `pgsty/mc`), #116 (scan Trivy de l'image).

Décision finale : construire l'image depuis **notre fork**, publiée sur GHCR (GitHub Container Registry), avec **tag explicite, jamais `latest`**. Raisons données par le projet : le fork garde le code même si l'amont disparaît ; un tag épinglé rend le build reproductible. Licence AGPL-3.0 (Affero General Public License) : le fork public publie les sources de l'image.

Leçon à formuler : une image tierce est une **dépendance de la chaîne d'approvisionnement** (supply chain) qui peut disparaître ou devenir privée sans préavis, et aucun outil automatique ne l'aurait signalé ici, d'où la procédure manuelle.

Mettre à jour l'image

1. Dans le fork : *Sync fork*.
2. Lancer le workflow *Publish GHCR image (DataShare)* avec le tag `RELEASE.*` voulu.
3. Reporter le nouveau tag aux 3 endroits, lancer la CI, merger.

À faire après chaque avis de sécurité de SILO. Limite : le workflow du fork ne construit que `linux/amd64` . Un scan Trivy en CI (non bloquant) remonte 4 CVE (Common Vulnerabilities and Exposures) `HIGH` sans correctif ; `silo` ne sert qu'en dev, CI et tests (voir `SECURITY.md`).

Dans DataShare

- Services : `deploy/docker-compose.yml` et `deploy/README.md` .
- Images applicatives : `backend/Dockerfile` , `frontend/Dockerfile` , `frontend/nginx.conf` .
- Procédure et historique : `MAINTENANCE.md` §1 (images tierces) et §2 (runbook).
- Aucune valeur par défaut dans `application-prod.yml` : variable manquante = démarrage refusé.
- Il n'existe pas de `docker-compose.prod.yml` (dit dans `SECURITY.md`).

À retenir

1. Le compose ne contient que les dépendances (PostgreSQL, stockage S3, bucket, Adminer) ; back et front ont leurs Dockerfile mais tournent hors compose.
2. Builds multi-étapes et utilisateur non root : images finales petites, surface réduite ; `service_healthy` garantit qu'un service est prêt et pas seulement démarré.
3. Dependabot ne couvre que les Dockerfile du projet : les images d'infrastructure sont suivies à la main et alignées sur trois fichiers.
4. MinIO a disparu de Docker Hub, puis de quay.io, puis de pgsty : d'où le fork épinglé sur GHCR. Une image tierce est une dépendance qui peut disparaître.
5. Limites assumées : `adminer:latest` non épinglé, fork `amd64` seulement, 4 CVE `HIGH` ouvertes sur `silo` , pas de déploiement prod conteneurisé complet.